

EC3404: GUI PROGRAMMING

Mini Project Report

SHOPVUE - MODERN E-COMMERCE SINGLE PAGE APPLICATION

Developed using Vue 3, TypeScript, Vite, Tailwind CSS, Pinia and DummyJSON

Name	Ahamad M.I.I.
Registration No.	EG/2023/5467

Abstract

ShopVue is a data-driven e-commerce Single Page Application (SPA) developed for the CS3404 GUI Programming mini project. The application uses Vue 3 with the Composition API, TypeScript, Vite and Tailwind CSS, and obtains product and authentication data from the public DummyJSON REST API. The project demonstrates component-based front-end architecture, strictly typed data models, asynchronous API consumption, reactive state management, reusable UI components, responsive design and client-side routing. In addition to the mandatory requirements, the implementation includes bonus-level features such as authentication simulation, a persistent shopping cart using Pinia, dynamic routing for product detail pages and a persistent light/dark theme. This report describes the architecture, implementation, data flow, user interactions, type-safety strategy, state management and requirement coverage of the submitted project.

Contents

1. Project Overview and Objectives
2. Requirement Compliance
3. Technology Stack
4. System Architecture
5. TypeScript and Type Safety
6. REST API Integration and Asynchronous Data Handling
7. Implemented Features
8. Component and View Design
9. State Management and Persistence
10. Routing and SPA Navigation
11. Tailwind CSS, Responsiveness and UI/UX
12. Application Data and Interaction Flows
13. Error Handling, Loading States and User Feedback
14. Build Configuration and Project Execution
15. Version Control and GenAI Deliverables
16. Testing and Verification Checklist
17. Current Scope and Future Improvements
18. Conclusion
- Appendix A - Project File Structure
- Appendix B - Key TypeScript Interfaces and Routes

1. Project Overview and Objectives

1.1 Project Context

The project requirement is to build a modern, data-driven SPA using Vue 3, TypeScript, Vite and Tailwind CSS while consuming data from DummyJSON. The chosen domain for this implementation

is an e-commerce store named ShopVue. The application is structured as a collection of views, reusable components, composables, Pinia stores and strict TypeScript interfaces rather than a single large component.

1.2 Main Objectives

- **Build a modern SPA:** Deliver navigation and page-like views without full browser page reloads.
- **Use strict TypeScript:** Model API data and application state using interfaces and typed functions instead of application-level any types.
- **Consume REST APIs:** Retrieve products, categories, individual product data and authentication responses from DummyJSON.
- **Create reusable components:** Separate product cards, filters, navigation, cart items, loading indicators and notifications into logical components.
- **Support user interaction:** Provide search, category filtering, price filtering, sorting, product detail viewing and cart operations.
- **Apply responsive styling:** Use Tailwind utility classes and responsive breakpoints for mobile, tablet and desktop layouts.
- **Implement advanced features:** Provide authentication simulation, global cart state with persistence, dynamic routing and dark mode.

1.3 Application Summary

A user can open the ShopVue home page, browse featured products and categories, navigate to the full product catalogue, search or filter the catalogue, open a product detail route, choose a quantity and add products to a persistent cart. The cart calculates the original value, discounted total and savings. The application also provides a login form connected to DummyJSON authentication and a theme toggle that persists the selected light/dark preference.

2. Requirement Compliance

The following matrix maps the supplied project requirements to the implementation found in the submitted source code.

Requirement	Status	Implementation Evidence
Vue 3 Composition API	Implemented	All major Vue files use <code><script setup lang="ts"></code> with <code>ref</code> , <code>computed</code> , <code>watch</code> and <code>onMounted</code> .
TypeScript	Implemented	Application files are TypeScript-based and interfaces are centralized in <code>src/types/index.ts</code> .
Vite	Implemented	Vite is the development/build tool; <code>vue()</code> and <code>tailwindcss()</code> plugins are configured.
Tailwind CSS	Implemented	Tailwind v4 is integrated through <code>@tailwindcss/vite</code> and <code>@import "tailwindcss"</code> .

No application-level any types	Implemented	Auth, products, filters and cart data use defined types/interfaces. The only any occurrence is in the generated vite-env declaration.
Reusable component architecture	Implemented	UI is split across six reusable components plus five route views.
Responsive layout	Implemented	Tailwind sm:, md: and lg: breakpoints are used throughout navigation, grids, detail and cart layouts.
Fetch API data	Implemented	Products/categories/detail use Axios; authentication uses Fetch.
Search or filtering	Implemented	Search, category, price range and sorting are provided.
Item detail view	Implemented	Dynamic /products/:id route renders ProductDetail.vue.
Authentication bonus	Implemented	POST /auth/login, token state and persisted authentication store.
Cart/persistence bonus	Implemented	Pinia cart store with localStorage persistence through pinia-plugin-persistedstate.
Dynamic routing bonus	Implemented	Vue Router dynamic product route and lazy-loaded route components.
Dark mode bonus	Implemented	Theme composable stores preference and toggles the dark class on the document root.

3. Technology Stack

Technology	Version / Source	Role in Project
Vue 3	^3.5.32	UI framework and reactive component model using the Composition API.
TypeScript	~6.0.2	Static typing for API models, component props, store state and function signatures.
Vite	^8.0.4	Development server and production bundler.
Tailwind CSS	^4.2.2	Utility-first responsive styling and custom design tokens.
Vue Router	^4.6.4	Client-side SPA navigation, dynamic product route and lazy-loaded views.
Pinia	^3.0.4	Global state management for cart and authentication.
pinia-plugin-persistedstate	^4.7.1	Persists selected Pinia store data to localStorage.

Axios	^1.15.0	Typed HTTP GET requests for products, categories and product details.
DummyJSON	Public REST API	External data source for products, categories and simulated authentication.

4. System Architecture

4.1 High-Level Architecture

The application uses a layered front-end architecture. App.vue acts as the application shell. Vue Router selects the active view. Views compose reusable components and call composables or Pinia stores. Product and authentication logic communicate with DummyJSON, while cart, authentication and theme preferences are persisted in browser localStorage.

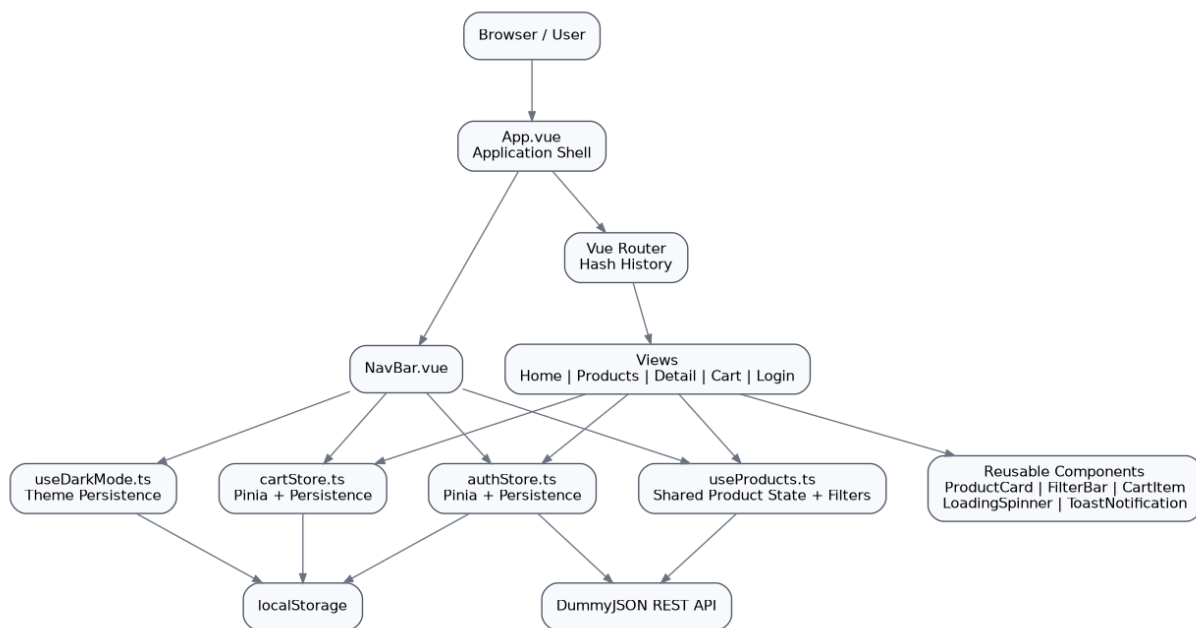


Figure 1. High-level component and data architecture of ShopVue.

4.2 Application Bootstrap

The runtime starts in src/main.ts. A Pinia instance is created, the persisted-state plugin is registered, and the Vue application is created from App.vue. Pinia and Vue Router are installed before the application is mounted to the #app element in index.html.

```
const pinia = createPinia()
pinia.use(piniaPluginPersistedstate)
createApp(App).use(pinia).use(router).mount('#app')
```

4.3 App Shell

- **NavBar.vue:** Permanent top navigation, search, theme toggle, cart indicator and authentication controls.
- **router-view:** Displays the currently matched view without performing a full page reload.

- **Page transition:** Uses Vue transition classes for smooth view changes.
- **Footer:** Provides project branding and submission information.
- **ToastNotification.vue:** Reusable notification infrastructure is mounted through a toast collection and dismiss event.

5. TypeScript and Type Safety

5.1 Centralized Data Models

The file `src/types/index.ts` centralizes the interfaces used by API responses, stores and components. This avoids repeatedly redefining data shapes and enables the TypeScript compiler to validate field access and function parameters.

Interface / Type	Purpose
ProductDimensions	width, height, depth
ProductReview	rating, comment, date, reviewer name/email
ProductMeta	created/updated timestamps, barcode, QR code
Product	complete product model including pricing, stock, images, tags, reviews and policies
ProductsResponse	products array plus total, skip and limit pagination metadata
LoginRequest	username, password and optional expiresInMins
LoginResponse	authenticated user profile, token and refreshToken
CartItem	Product plus quantity
ProductFilters	search, category, sort mode and price range
AsyncState<T>	generic data/loading/error state

5.2 Strict Typed Operations

- **Typed Axios responses:** `axios.get<ProductsResponse>(...)` and `axios.get<Product>(...)` bind the expected API response shape.
- **Typed props:** `ProductCard` receives `product: Product`; `CartItem` receives `item: CartItem`; `ProductDetail` receives `id: number`.
- **Typed emitted event:** `ToastNotification` defines a dismiss event signature.
- **Typed filter updates:** `setFilter<K extends keyof ProductFilters>(key, value)` ensures the value type matches the selected filter key.
- **Union types:** `sortBy` accepts only `'price-asc'`, `'price-desc'`, `'rating'` or `'default'`; toast and star states also use restricted string unions.
- **Generic state:** `AsyncState<T>` is reusable for asynchronously loaded data.

The authored application code does not rely on loose any types for product, cart or authentication data. A generated Vite declaration file contains a framework-level any, but this is not application business logic.

6. REST API Integration and Asynchronous Data Handling

6.1 API Endpoints Used

Endpoint	Used In	Purpose
GET /products	useProducts.ts	Loads product batches using limit, skip and a selected field list.
GET /products/categories	useProducts.ts	Loads category metadata and stores category slugs.
GET /products/:id	useProducts.ts	Loads the full details of one product for the detail view.
POST /auth/login	authStore.ts	Authenticates a DummyJSON user and receives access/refresh token data.

6.2 Product Fetch Strategy

The product composable keeps shared module-level reactive state so that different views can access the same loaded catalogue. `fetchAllProducts()` uses a limit of 100 records per request, increments skip after each batch and combines each response into the existing array. The current loop includes a protective cap that stops after loading up to 200 products. An initialized flag prevents the catalogue from being refetched unnecessarily during later calls in the same session.

6.3 Asynchronous State

The shared `AsyncState<Product[]>` object contains data, loading and error properties. Before a product fetch, loading becomes true and the old error is cleared. The request is executed inside `try/catch/finally`. Errors are converted into a user-readable message, and finally guarantees that loading returns to false regardless of success or failure. `HomeView` and `ProductsView` start the product and category requests in parallel using `Promise.all`.

6.4 Axios and Fetch

Two HTTP approaches are intentionally present in the project. `Axios` is used for product GET requests because it directly exposes parsed JSON through `response.data` and supports generic response typing. The native `Fetch` API is used for the authentication POST request, where the implementation manually sets `Content-Type`, serializes the request body, checks `response.ok` and parses JSON.

7. Implemented Features

7.1 Home Page

- Automatic rotating hero banners with a five-second interval and manual banner dots.
- Primary navigation buttons to the product catalogue and login page.
- Statistics for product count, category count, displayed average rating and number of items in the cart.

- Quick category buttons for the first twelve API categories.
- Top-rated product section based on products with a rating of at least 4.5, with up to eight products displayed.
- Loading and API error states while catalogue data is being retrieved.

7.2 Product Catalogue, Search, Filtering and Sorting

- **Search:** The shared search text is matched case-insensitively against product title, brand and description.
- **Category filter:** The catalogue can be restricted to an exact DummyJSON category slug.
- **Price range:** Minimum and maximum price values are applied to each product.
- **Sorting:** Products can be sorted by ascending price, descending price or highest rating.
- **Reset:** Active category, sort and price filters can be restored to default values.
- **Pagination:** Client-side pagination displays 20 filtered products per page and shows nearby page numbers with previous/next controls.
- **Empty state:** A dedicated message is shown when no products match the selected filters.

7.3 Product Cards

- Thumbnail image with lazy loading and hover zoom.
- Category, title, brand, star rating and discounted price display.
- Discount badge when discount percentage is greater than 5%.
- Out-of-stock overlay and disabled add-to-cart control for unavailable products.
- Immediate Add to Cart feedback and an Add More state when the item is already present.
- Dynamic route links from both image and title to the product detail view.

7.4 Product Detail View

- Dynamic route parameter receives the product id as a number.
- Main product image and selectable image thumbnails.
- Category, brand, title, rating, review count, description and tags.
- Computed discounted price and amount saved.
- Stock and availability information.
- Quantity controls bounded between 1 and available stock.
- Add-to-cart action with temporary success state.
- Shipping, return, warranty and minimum-order information from the API.
- Customer review cards with reviewer name, date, rating and comment.
- Dedicated loading and product-not-found states.

7.5 Shopping Cart

- Global cart store shared across pages.
- Add products and increase an existing item quantity while respecting stock limits.
- Decrease quantity, remove individual items and clear the complete cart.
- Persistent cart items across page reloads.

- Computed total item count, original subtotal, discounted total and savings.
- Order summary with free shipping display and savings feedback.
- Responsive empty-cart state and Continue Shopping navigation.
- Checkout control is currently a front-end placeholder and does not create an order on a backend.

7.6 Authentication Simulation

- Login form with username and password bound using v-model.
- Show/hide password control.
- POST request to DummyJSON /auth/login with expiresInMins set to 60.
- Loading state while the login request is active.
- Error display when DummyJSON rejects the credentials.
- Authenticated user data and token stored in a Pinia store and persisted to localStorage.
- Navigation bar switches between Login and authenticated user/logout controls according to computed isAuthenticated state.

7.7 Dark Mode

useDarkMode.ts determines the initial theme from the stored shopvue-dark-mode value. If no preference has been stored, it uses the browser operating-system preference through matchMedia. A watcher calls applyDark whenever isDark changes. The function adds or removes the dark class on document.documentElement and saves the selected state to localStorage. Tailwind dark: modifiers then apply the alternative visual styles.

8. Component and View Design

8.1 Reusable Components

Component	Responsibility
NavBar.vue	Global navigation, search, dark-mode toggle, cart count, auth status and mobile menu.
ProductCard.vue	Reusable product summary card and add-to-cart control.
FilterBar.vue	Sort control, price inputs, category pills and filter reset.
CartItem.vue	Individual cart line with thumbnail, pricing, quantity update and removal.
LoadingSpinner.vue	Typed reusable spinner supporting sm/md/lg sizes and an optional message.
ToastNotification.vue	Dismissible timed notification component with success/error/info variants.

8.2 Route Views

View	Responsibility
------	----------------

HomeView.vue	Landing experience, hero, categories and featured products.
ProductsView.vue	Full catalogue, filters and 20-item client-side pagination.
ProductDetail.vue	Single-product detail, quantity, cart action and reviews.
CartView.vue	Cart list, summary calculations and cart actions.
LoginView.vue	Authentication form and login state feedback.

9. State Management and Persistence

9.1 Product State - Composable

Product catalogue state is handled by useProducts.ts rather than Pinia. Its refs are declared at module level, so every call to useProducts() shares the same allProducts, categories, totalProducts, filters and async state. This gives simple singleton-like shared state while keeping product-specific logic in a dedicated composable.

9.2 Cart Store - Pinia

cartStore.ts uses Pinia setup-store syntax. The items array is the source state. totalItems, totalPrice, originalPrice and savings are computed values derived from items. Store methods mutate the cart through clear operations rather than allowing components to duplicate cart logic. The persisted-state plugin writes only items to localStorage using the key shopvue-cart.

9.3 Authentication Store - Pinia

authStore.ts maintains user, token, loading and error state. isAuthenticated is computed from token presence, while fullName is computed from the user record. The persisted-state plugin stores user and token using the key shopvue-auth. The token is also written to the auth_token localStorage key during login and removed during logout.

10. Routing and SPA Navigation

Vue Router is configured with createWebHashHistory(), which represents routes after a # fragment. This is suitable for static hosting because the server only needs to serve the main index file. HomeView is imported eagerly, while the remaining route views use dynamic imports and are therefore eligible for lazy loading.

Path	Route Name	Destination / Behavior
/	home	HomeView.vue
/products	products	ProductsView.vue
/products/:id	product-detail	ProductDetail.vue; id converted to Number and passed as a prop
/cart	cart	CartView.vue
/login	login	LoginView.vue

/:pathMatch(.*)*	-	Redirect to / for unknown routes
------------------	---	----------------------------------

The router also defines `scrollBehavior()`. If the browser has a saved scroll position, it restores it; otherwise it returns the page to the top. The `App.vue` router-view uses a Vue transition to animate route changes.

11. Tailwind CSS, Responsiveness and UI/UX

11.1 Tailwind v4 Integration

Tailwind CSS is integrated as a Vite plugin in `vite.config.ts` and imported from `src/style.css`. The stylesheet uses `@theme` to define primary and accent color tokens and a custom Inter font stack. Reusable component-level utility compositions are declared inside `@layer` components.

```
@theme {
  --color-primary-500: #3b82f6;
  --color-primary-600: #2563eb;
  --color-accent-500: #ec4899;
  --font-sans: 'Inter', system-ui, sans-serif;
}
```

11.2 Reusable Visual Classes

- `btn-primary` for emphasized actions.
- `btn-secondary` for bordered secondary actions.
- `btn-ghost` for low-emphasis controls.
- `card` for reusable rounded surface styling and hover elevation.
- `input` for consistent form fields and focus states.
- `badge` for pills such as categories and discounts.
- `section-title` for responsive section headings.

11.3 Responsive Design

- **Navigation:** Desktop links/search are hidden below the `sm` breakpoint, where a collapsible mobile menu and mobile search are used instead.
- **Product grid:** Changes from one column to two at `sm`, three at `md` and four at `lg`.
- **Category grid:** Scales from two columns through `sm/md` to six columns at `lg`.
- **Product detail:** Single-column layout becomes a two-column image/information layout at `lg`.
- **Cart:** Stacks vertically on smaller screens and becomes a side-by-side items/summary layout at `lg`.
- **Spacing:** Responsive `px` and `py` classes adapt page padding and hero spacing for different screen widths.

11.4 Interaction and Accessibility Details

- Hover and active states on cards, buttons and navigation controls.
- Page, toast, hero, cart-item, cart-count and mobile-menu transitions.
- Image alt text on product and user images.

- ARIA labels on icon-only controls such as the theme toggle, cart button, mobile menu and remove-item control.
- Visible disabled states for stock limits and loading actions.
- Dark-mode colors applied throughout the major views and reusable classes.

12. Application Data and Interaction Flows

Vue reactivity connects user input, application logic and screen rendering. A user action changes reactive state through a component, composable or Pinia store. Computed properties derive new values from this state, and Vue updates only the relevant rendered output. API-backed operations insert an asynchronous HTTP step into the same flow.

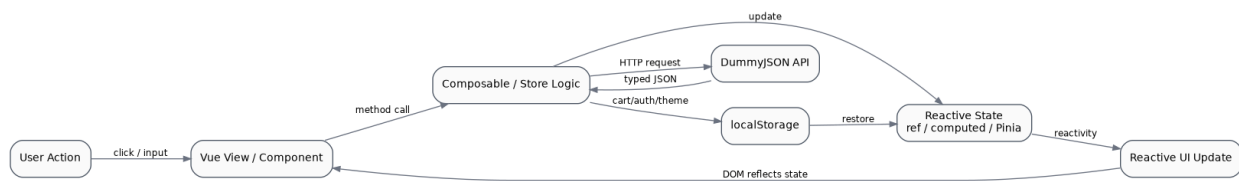


Figure 2. General user-action, API, state and reactive-rendering flow.

12.1 Product Search Flow

1. The user enters text in the NavBar search input.
2. v-model updates searchQuery.
3. Submitting the form calls `setFilter("search", searchQuery)`.
4. If the current route is not Products, the router navigates to `/products`.
5. `filteredProducts` recomputes automatically and applies the search to title, brand and description.
6. `ProductsView` paginates the filtered array and `ProductCard` components re-render the matching results.

12.2 Add-to-Cart Flow

7. The user clicks Add to Cart from a product card or product detail page.
8. The component calls `cart.addToCart(product, quantity)`.
9. The store checks whether the product already exists in the cart.
10. The quantity is added or a new `CartItem` is inserted, with the quantity capped by available stock.
11. Computed totals recalculate automatically.
12. The navigation cart badge and `CartView` update reactively.
13. `pinia-plugin-persistedstate` saves the items array so it can be restored after reload.

12.3 Login Flow

14. The user submits the `LoginView` form.
15. `auth.login(username, password)` sets `isLoading` and clears the old error.
16. `Fetch` sends a JSON POST request to `DummyJSON /auth/login`.
17. If the response is not successful, the returned error message is thrown and displayed.
18. If successful, `LoginResponse` data is assigned to `user` and `token`.

19. The token is persisted and isAuthenticated becomes true automatically.
20. NavBar reacts by displaying the authenticated user controls and logout option.

13. Error Handling, Loading States and User Feedback

- **Product loading:** HomeView and ProductsView show LoadingSpinner while the shared product request is active.
- **Product-list failure:** useProducts records an error string; views render a red error state instead of failing silently.
- **Product detail failure:** fetchProduct returns null when the request fails, producing a Product not found state with a navigation option.
- **Authentication failure:** Non-2xx responses are converted into an Error using the API message and shown inside LoginView.
- **Authentication loading:** The submit button is disabled and displays a spinner while authentication is in progress.
- **Cart feedback:** Add buttons temporarily change their text/state, stock-limited controls are disabled, and the cart shows an empty state when appropriate.
- **Toast infrastructure:** ToastNotification supports timed success/error/info messages and emits dismiss, providing a reusable mechanism for future global notifications.

14. Build Configuration and Project Execution

14.1 Vite Configuration

vite.config.ts registers two plugins: @vitejs/plugin-vue for Vue Single File Components and @tailwindcss/vite for Tailwind CSS processing. No custom server or backend build configuration is required because the application is entirely client-side and communicates with an external API.

14.2 Available npm Scripts

Command	Purpose
npm run dev	Starts the Vite development server.
npm run build	Runs vue-tsc type checking/build step and creates a production Vite build.
npm run preview	Serves the production build locally for preview.

14.3 Installation and Run Instructions

```
npm install
npm run dev
```

For a production verification build, run:

```
npm run build
```

The project intentionally excludes `node_modules` from the deliverable. Dependencies are recreated from `package.json/package-lock.json` by running `npm install`.

15. Version Control and GenAI Deliverables

15.1 Git and GitHub

The assignment requires a public GitHub repository link so that commit history and development progress can be assessed independently from the ZIP submission. The project source is maintained in a GitHub repository, and the repository link should be submitted alongside the ZIP. Good practice for this project is to keep meaningful, focused commits such as implementing product fetching, adding filtering, creating the cart store, introducing routing or refining responsive styling.

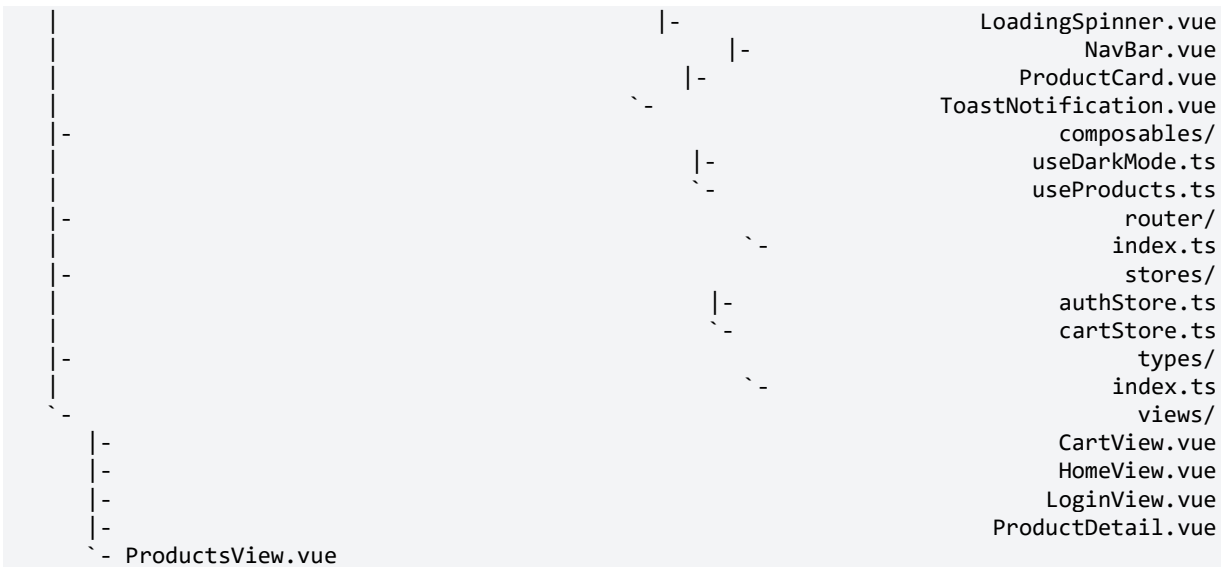
15.2 GenAI Usage Log

The project specification permits the use of generative AI for tasks such as CSS class generation, debugging and TypeScript interface assistance, but requires the student to understand the resulting code. A separate `prompts.txt` file should accompany this report and preserve the actual prompts used during development in the required Aim / Actual prompt format. The report itself does not invent prompts that are not present in the submitted source material.

16. Testing and Verification Checklist

The following checklist can be used before the final submission and viva to verify the user-visible behavior of the application.

- ☐ Application starts successfully with `npm run dev` and no blocking console error appears.
- ☐ Home page loads products and categories from DummyJSON.
- ☐ Hero banner changes automatically and manual banner dots work.
- ☐ Product catalogue displays results and pagination controls work.
- ☐ Search matches title, brand or description.
- ☐ Category and price filters update the visible catalogue.
- ☐ Sort options reorder products by price or rating.
- ☐ Product card opens `/products/:id` and detail information is loaded.
- ☐ Quantity controls do not exceed stock or go below one.
- ☐ Add to Cart updates the navigation count and CartView.
- ☐ Cart quantity update, remove and clear operations work.
- ☐ Cart survives a page reload due to persistence.
- ☐ Valid DummyJSON credentials update the authenticated state and invalid credentials show an error.
- ☐ Logout clears the active authentication state.
- ☐ Dark mode toggles and remains selected after a page reload.
- ☐ Layout is checked at mobile, tablet and desktop widths.



Note: The root script.js file present in the ZIP is not referenced by index.html; the active application entry point is /src/main.ts.

Appendix B - Key TypeScript Interfaces and Routes

B.1 Core Response Shapes

```

interface ProductsResponse {
  products: Product[]
  total: number
  skip: number
  limit: number
}

interface CartItem {
  product: Product
  quantity: number
}

interface AsyncState<T> {
  data: T | null
  loading: boolean
  error: string | null
}

```

B.2 Filter Type

```

interface ProductFilters {
  search: string
  category: string
  sortBy: 'price-asc' | 'price-desc' | 'rating' | 'default'
  minPrice: number
  maxPrice: number
}

```

B.3 Dynamic Product Route

```

{
  path: '/products/:id',
  name: 'product-detail',
  component: () => import('../views/ProductDetail.vue'),
}

```

```
props: (route) => ({ id: Number(route.params.id) })
```